AutoFuzzer: A Standalone ESP32-Based UART Fuzzing Platform for Embedded Firmware Security and Robustness Testing

Abishek R Nair
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham, Amritapuri Campus
Kerala, India
AM.EN.U4EAC24004

Alwin Varghese
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham, Amritapuri Campus
Kerala, India
AM.EN.U4EAC24009

Dr. Rajesh Kannan Megalingam
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India

***Abstract*—Serial-communication parsers in microcontroller firmware form an exposed attack surface: malformed, oversized, truncated, or corrupted packets can trigger hangs, resets, or memory faults, yet such parsers are usually tested only with well-formed traffic. This paper presents AutoFuzzer, a low-cost standalone fuzzing platform in which an ESP32 acts as fuzzing controller and liveness monitor for a UART-connected device under test (DUT). The controller generates seven primary classes of valid and malformed frames, schedules them in timed campaign phases, interprets ACK/NACK responses, and watches a DUT heartbeat pin for hangs and resets. A per-packet seeded xorshift32 generator enables deterministic replay; failures are classified, replayed, minimized, and summarized as a 0–100 robustness score. The platform runs from an OLED/button interface or a serial command interface, without a PC. The evaluation is deliberately limited: 45 host-side tests of a Python reference implementation pass, and one recoverable UART run (258 packets, 14 NACKs, no failures, 75/100) is consistent with the firmware scoring rules, although its target and raw log are unavailable. No vulnerability discovery is claimed, fault-detection effectiveness remains unproven, and the 41 archived reports are synthetic test data.**

***Index Terms*—cybersecurity, firmware security, fuzz testing, embedded systems, UART, ESP32, heartbeat monitoring, hardware-in-the-loop testing**

I. Introduction

Embedded devices exchange data with sensors, peripherals, and other controllers over UART, SPI, I2C, and CAN, and the firmware drivers that parse these byte streams form an input-validation boundary. A parser that mishandles an empty, oversized, truncated, or corrupted frame can hang, reset, or corrupt memory. These availability and integrity failures can be triggered deliberately by an attacker with access to the interface, or accidentally by a faulty peer. Fuzzing, which feeds malformed input to a system and watches for crashes or hangs, dates back to [1] and is surveyed in [2], [3].

Functional testing shows that a parser accepts valid frames but says little about invalid ones. Fuzzing embedded devices is harder than fuzzing desktop software because memory corruption is hard to observe and tooling differs across platforms [4]. Most embedded fuzzing literature addresses emulation, rehosting, and instrumentation, with less attention to input generation and scheduling.

Threat model and scope. We assume an adversary or faulty peer that can inject arbitrary byte sequences on a UART link, and we target failures observable without instrumentation: hangs, resets, watchdog events, and protocol errors. AutoFuzzer is a cooperative black-box bench tool. It needs a serial connection, a shared frame protocol, and a heartbeat pin on the DUT, and it cannot see silent memory corruption that leaves the main loop alive. It is not a universal instrumentation framework.

AutoFuzzer sends mutated frames from an ESP32 DevKit V1 to the DUT over UART and analyses the DUT responses and heartbeat, with no PC or target-firmware changes beyond the serial link and heartbeat. The contributions are:

- A standalone ESP32 controller with OLED and buttons that runs campaigns without a computer.
- A framed UART test protocol with ten mutation types (seven primary classes, Table I) and a seeded, replayable scheduler (Quick, Standard, Deep, Custom).
- A heartbeat-plus-response failure oracle with classification, replay, minimization, and a weighted robustness score.
- An initial hardware assessment that states what it shows and what it does not.

Replay and minimization are implemented in the firmware, but their results are not reported (Section III-F).

II. Related Work

Fuzzing and protocol fuzzing. Fuzzing techniques are reviewed in [2], and recent work surveys the statefulness and message-structure issues of protocol fuzzing [3]. AFLNet is a grey-box network-protocol fuzzer [6], but it targets server applications on general-purpose computers.

Emulation and rehosting. Because real hardware offers limited observability, many embedded fuzzers run firmware in an emulator. HALucinator replaces hardware abstraction layer APIs [7], P2IM models peripherals automatically [8], and Pemu generates valid packets for embedded network stacks [9]. A survey of bare-metal firmware fuzzing notes that input generation and scheduling have received far less attention than emulation and instrumentation [5].

On-target and hardware-assisted fuzzing. SHiFT fuzzes firmware natively through semihosting, with up to two orders of magnitude speedup over software emulation [10]. EmbedFuzz transplants firmware to a fast host and reports up to eight times the throughput of existing microcontroller fuzzers [11]. µAFL obtains coverage without instrumenting the target [12], GDBFuzz uses debug ports and hardware breakpoints [13], a JTAG-based framework targets ESP32 coverage [14], and TaPaFuzz attaches a hardware monitor to a RISC-V core on an FPGA, reporting an 11.7× speedup over emulation [15].

Oracles, reproduction, and evaluation. Embedded systems often lack memory protection, so bugs do not always crash the device [4]. µSBS exposes silent corruption by instrumenting bare-metal binaries [16], and Werquin et al. use the physical reactions of CAN control units as oracles [17]. Delta debugging reduces a failing input to a minimal form [18], and a review of 32 fuzzing studies found problems in all of them and gave guidelines for valid experiments [19].

Gap and positioning. The embedded fuzzing work considered here centers on coverage-guided fuzzing enabled by emulation, rehosting, debugging, or FPGA assistance [5], [9]–[11], [13], [15]. Within the cited works, lightweight robustness testing of a serial frame parser with a standalone, inexpensive fuzzer receives less attention. The review is not exhaustive and no claim of priority is made. AutoFuzzer trades coverage information for simplicity, low cost, and platform independence, and inherits the limits of a liveness oracle (Section VII).

III. System Design and Methodology

A. Overview and Frame Format

AutoFuzzer separates the fuzzing controller from the DUT (as depicted in Fig. 1). The role of the fuzzing controller is to send out and transmit frames, receive answers, and observe the heartbeat signal of the DUT. The reference firmware running on the DUT receives frames and generates ACK/NACK signals upon acceptance/rejection of frames, as well as sends heartbeat signals periodically through an individual pin. This design assumes that there is a shared ground and logic levels (refer to Section VIII).

TABLE I
Seven Primary Mutation Classes and Intended Reference-DUT Response

Class	Payload	Construction	Intended response
VALID	8 B	Well-formed (baseline)	ACK
EMPTY	0 B	Zero length	n/r
MAX_LEN	32 B	Largest valid payload	ACK
OVERLENGTH	45 B	Declared length > 32	NACK `0x02`
BAD_CHECKSUM	8 B	Inverted checksum	NACK `0x03`
TRUNCATED	12→6 B	Half of bytes sent	No response
RANDOM	1–32 B	PRNG-selected payload	n/r

n/r: not reported.

TABLE II
Security Weakness Classes Targeted by Each Mutation (Intended, Not Demonstrated)

Mutation	Weakness probed	CWE
EMPTY, RANDOM	Unexpected/degenerate input	CWE-20, 755
OVERLENGTH	Missing length check, buffer overflow	CWE-120
BAD_CHECKSUM	Unchecked integrity value	CWE-354
TRUNCATED	Incomplete frame, parser stall	CWE-835

The checksum is the bitwise XOR of all bytes from COMMAND through the last payload byte. With n the payload length and p_i the payload bytes,

$$c = \mathrm{CMD} \oplus \mathrm{LEN} \oplus \mathrm{SEQ_}L \oplus \mathrm{SEQ_}H \oplus \bigoplus_{i=0}^{n-1} p_i. \quad (1)$$

A test frame consists of SYNC (`0xA5`), CMD (`0x01`), LEN, SEQ_L, SEQ_H, the payload, and the XOR byte. The reference DUT accepts payloads of up to 32 bytes. Its response consists of the sync byte `0x5A`, the echoed 16-bit sequence number, and a status byte: `0x00` (ACK), `0x02` (NACK, overlength payload), or `0x03` (NACK, checksum error). Absence of a response is interpreted as a parser timeout.

B. Mutation Classes

The firmware defines ten mutation types; the assessment focuses on the seven primary types in Table I. The host test suite covers these seven. The other three are BAD_HEADER (corrupted synchronization byte), BAD_LEN (length field 0xFF), and SEQ_JUMP (sequence number jump), each intended to make the protocol reject the message. The bad-checksum mutation (MUT_BAD_CRC in the firmware) transmits the bitwise complement $\bar{c}$ of the checksum in Eq. (1), which is always wrong.

Each class probes a common input-handling weakness (Table II). The mapping to MITRE CWE categories states what each class is designed to exercise; it does not claim that any such weakness was found in a DUT.

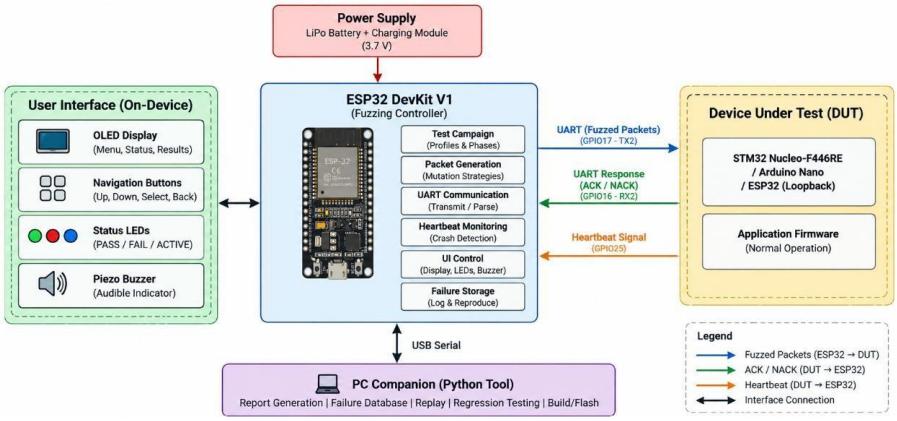


Fig. 1. System architecture of AutoFuzzer: the ESP32 controller sends fuzzed packets over UART (GPIO17), receives ACK/NACK responses (GPIO16), and monitors the DUT heartbeat (GPIO25); an OLED, buttons, status LEDs, and buzzer form the on-device interface, and an optional PC companion connects over USB serial.

C. Campaign Scheduling

A campaign profile fixes the duration: Quick (30 s), Standard (60 s), Deep (5 min), or Custom. A mutation class defines how one packet is built, whereas a campaign phase is a time window that selects which classes are sent. The firmware supports BASELINE (valid packets), BOUNDARY (empty, maximum length), OVERLENGTH, CHECKSUM, MALFORMED, RANDOM, and FREEFORM phases. The Quick profile runs the first six for 5 s each (0–30 s); FREEFORM starts at 30 s and applies only to the longer profiles. Per-class counts of NACKs, timeouts, and heartbeat failures raise the selection weights of the corresponding mutations.

D. Deterministic Reproduction

Mutation randomness comes from an xorshift32 generator [20] with a non-zero seed and deterministic state updates. For each packet the firmware stores the seed, the PRNG state before and after generation, the sequence number, the mutation class, and the full payload. Restoring the PRNG state and mutation class regenerates an identical packet, which was checked for all seven classes.

E. Failure Detection and Classification

Until a heartbeat edge is detected, the controller stays in a protocol-check state, so no campaign runs against a disconnected DUT. Let t_{hb} be the timestamp of the last heartbeat edge and $T_{hb} = 350\,\text{ms}$ the timeout. The DUT is declared failed when

$$t - t_{hb} > T_{hb}. \tag{2}$$

Responses are matched to transmitted packets by sequence number and labelled ACK, NACK, or timeout. A timeout after a truncated frame is the expected parser behaviour unless the heartbeat also shows a loss of liveness. Ten failure types are defined (e.g., `HEARTBEAT_TIMEOUT`, `NO_RESPONSE`, `PARSER_TIMEOUT`, `PROTOCOL_ERROR`, `WATCHDOG_RESET`); an in-memory buffer holds up to 20 failure records and can be exported in a JSON-like format.

F. Replay and Minimization

A failure is replayed a user-set number of times (default three) to check consistency. Minimization shrinks the failing test case by a guided binary search in the spirit of delta debugging [18]. A debugging command (SIMFAIL) simulates a failure so both components can be exercised without a DUT.

G. Robustness Score

The firmware (`core/result.cpp`) reports an integer score $S \in [0, 100]$ as a weighted sum of six category scores $s_k \in [0, 100]$ with weights w_k summing to 100,

$$S = \left\lfloor \tfrac{1}{100} \textstyle\sum_{k=1}^{6} w_k s_k \right\rfloor. \tag{3}$$

The defined weights (w_k) are communication (25), boundary (20), malformed input (20), recovery (15), heartbeat stability (10), and random input (10). Let A, N, P, and H be the numbers of ACKs, NACKs, transmitted packets, and heartbeat timeouts, respectively, and let f_c be the number of stored failure records in category c (boundary: VALID, EMPTY, MAX_LEN; malformed: OVERLENGTH, BAD_CHECKSUM; random: RANDOM). The communication score is $s_{\text{comm}} = \lfloor 100A/(A+N) \rfloor$, set to 0 if $A+N = 0$. The recovery score is $s_{\text{rec}} = \lfloor 100(P - H)/P \rfloor$, set to 0 if $P = 0$. For $c \in \{\text{boundary}, \text{malformed}, \text{random}\}$, the category score is $s_c = 100 - 33 f_c$ when $f_c \leq 3$, and 0 when $f_c > 3$. The heartbeat score is 0 if no heartbeat edge was ever observed; otherwise, it is 100, 75, 50, or 0 for $H = 0$, $H \leq 2$, $H \leq 5$, or $H > 5$, respectively. The verdict is FAIL if any failure was recorded; PASS if no failures were recorded, at least one packet was sent, and at least one ACK or NACK was received; and INCONCLUSIVE otherwise.

Three factors limit interpretation of the score. First, with no failures and a heartbeat present, $S = 75 + \lfloor s_{\text{comm}}/4 \rfloor$, so the score depends only on the ACK fraction, which reflects the share of valid frames in the traffic mix rather than DUT quality. Second, TRUNCATED and three other mutation types belong to no scoring category. Third, a single ACK or NACK suffices for a PASS verdict.

TABLE III
DUT WIRING TO ESP32 GPIO17 (TX), GPIO16 (RX), GPIO25 (HEARTBEAT)

DUT	RX (from 17)	TX (to 16)	HB (to 25)
STM32 Nucleo-F446RE	PA10	PA9	PB5
Arduino Nano	D3	D2	D4
ESP32 (loopback)	GPIO16	GPIO17	GPIO25

IV. IMPLEMENTATION

A. Hardware

The controller is an ESP32 DevKit V1 on a perfboard (Fig. 2, left) with an I2C OLED, three status LEDs (PASS green, FAIL red, ACTIVE blue), four buttons, a piezo sounder, and a LiPo battery with charger. Per the repository pinout, UART uses GPIO17/16, the heartbeat GPIO25, and the OLED GPIO21/22; other pins serve the LEDs, buttons, and sounder. DUT wiring is given in Table III.

B. Firmware Architecture and Interfaces

The ESP32 firmware comprises a core module (PRNG, campaign manager, test-case collector, error handler, scorer), a protocol module (UART fuzzer, parser, SPI and I2C stubs), a monitoring module (heartbeat monitor, crash detector), a user interface, and storage. Peripherals are serviced without blocking delays, and all buttons share a 25 ms debounce time. Reference DUT firmware exists for the Arduino Nano and STM32 Nucleo boards.

On the device, the OLED menu starts at NEW TEST and proceeds through target board, protocol, wiring diagram, and profile, then runs the campaign and opens a results menu (show results, show failures, replay failure, create report). The serial interface accepts START, STOP, PAUSE, STATUS, EXPORT, REPLAY, RESET, HBRAW (raw heartbeat pin data), and SIMFAIL. The repository also contains a Python PC utility (reporting, SQLite failure history, regression replay, PlatformIO build and flashing, a browser dashboard, and an AI debugging aid that needs human validation), which is outside the scope of this paper.

C. Software Verification

The host-side suite has 45 tests: PRNG (6), checksum (3), packet generation (8), mutation selection (5), response parsing (6), failure classification (6), scoring (6), reporting database (3), and replay (2). All pass. The suite is a Python reference implementation of the algorithms and does not run the ESP32 firmware, so passing it shows only the internal consistency of the algorithms and data processing, not the correctness of the firmware.

V. EXPERIMENTAL SETUP

Three DUT configurations were used: (i) an Arduino Nano (CH340 clone) with SoftwareSerial at 9600 baud (ACK/NACK responses at 115200 baud were not detected, so that rate is excluded); (ii) an ESP32 DevKit V1 in loopback (TX2 to RX2) at 115200 baud; and (iii) an STM32 Nucleo-F446RE using USART1 at 115200 baud. Controller and reference firmware were built with PlatformIO (espressif32@6.7.0, atmelavr, ststm32).

The user selects board, protocol (UART), and profile, wires the DUT as shown on the OLED, and the campaign starts once a heartbeat is detected. Each result comes from a single 30 s Quick campaign, so no variance is available. Reports are stored in the project repository.

TABLE IV
RECORDED RESULTS OF SINGLE QUICK CAMPAIGNS

Metric	Nano	ESP32 loop	Serial run
Baud rate	9600[b]	115200[b]	n/r
Duration	30 s[b]	30 s[b]	25.827 s
Packets sent	n/v	n/v	258
ACKs / NACKs	n/v	n/v	0 / 14
Failures	n/v	n/v	0 reported
Verdict	n/v	n/v	PASS reported
Score	n/v	n/v	75/100

[b]Values appear in the README but are not independently verified. n/r: not reported; n/v: not verifiable from recoverable records.

VI. RESULTS AND ANALYSIS

A. Single-Campaign Results

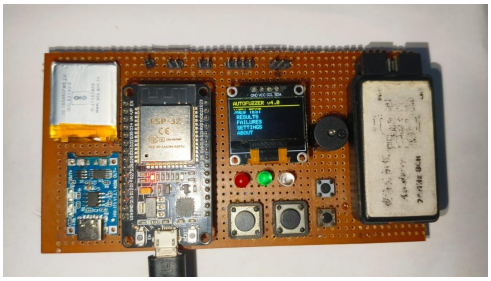
Table IV summarizes the recorded results. According to the project README, two 30 s campaigns produced 600 packets and scores of 94/100 (Arduino Nano) and 92/100 (ESP32). The original logs and firmware version are no longer available, and 600 packets in 30 s conflicts with the firmware's minimum 100 ms packet interval, which implies roughly 300 packets. These two campaigns are therefore treated as preliminary and excluded from all calculations. The loopback test exercises the controller's own transmit-receive path, not a separate DUT.

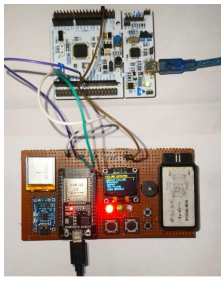
In a further UART Quick run captured on the serial monitor (Fig. 3), the fuzzer sent 258 packets in 25.827 s with 0 ACKs, 14 NACKs, no failures, and a score of 75/100. The scoring rules reproduce this value ($s_{\mathrm{comm}} = 0$, all other categories 100, so $S = 75$), but neither the target board nor the raw log is available. The log shows six phases, whereas the firmware adds FREEFORM from 30 s.

B. Synthetic Report Data

The repository holds 41 JSON reports (28 PASSED, 13 FAILED; 15 STM32 Nucleo, 18 Arduino Nano, 8 ESP32 loopback). Commit history shows they were generated as test data to exercise the reporting pipeline, not measured on hardware. Their logs are missing, their mutation distributions do not match packet counts, their scores are often not reproducible from the stated rules, and their schema differs from the report generator's. They are therefore not used to assess hardware performance or detection efficiency.

The development log records integration problems: unstable SoftwareSerial at 115200 baud, a mismatch between the wiring diagram and the pin mapping in code, a floating heartbeat line causing spurious edges (fixed by edge debouncing), and an ST-Link failure on one Nucleo board. No quantitative performance measurements have been made.





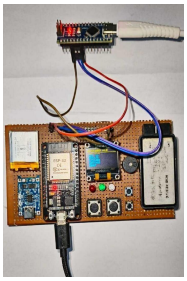


Fig. 2. Left: AutoFuzzer prototype (ESP32, OLED, LEDs, buttons, buzzer, battery, charging module on a perfboard). Centre: connected to an STM32 Nucleo-F446RE (the OLED shows the failure-handling menu). Right: connected to an Arduino Nano.

```
========================================
CAMPAIGN START: QUICK 30s profile
========================================
Phase -> BASELINE     (0-5s: valid packets)
Phase -> BOUNDARY     (5-10s: empty/max length)
Phase -> OVERLENGTH   (10-15s: oversized payloads)
Phase -> CHECKSUM     (15-20s: bad CRC)
Phase -> MALFORMED    (20-25s: truncated)
Phase -> RANDOM       (25-30s: random)

========================================
AUTOFUZZER TEST RESULT
========================================
Protocol: UART
Profile:  QUICK 30s
Duration: 25827 ms
Packets:  258
ACKs:     0
NACKs:    14
Failures: 0
ROBUSTNESS SCORE: 75/100
VERDICT: PASS
========================================
```

Fig. 3. UART Quick campaign output on the serial monitor (258 packets, 0 ACKs, 14 NACKs, 0 failures, score 75/100).

VII. Discussion

A. What the Results Demonstrate

The evidence shows that the project has a host-side reference test suite and a reporting procedure that reflects UART campaign results. The 45 passing host-side tests establish the consistency of the algorithms and data processing under the conditions of Section IV-C, and one recorded UART campaign illustrates the reporting and scoring flow. The evidence does not show that campaigns completed successfully on real hardware, and it does not show that the platform detects faults.

B. Trade-offs and Threats to Validity

The heartbeat oracle gives a low-cost, architecture-neutral implementation but tests liveness only. It can reveal hangs, resets, and lockups, but not silent memory corruption that leaves the main loop running [4], [16]. Coverage-guided fuzzers [10]–[13], [15] seek new execution paths but require instrumentation or debug access. AutoFuzzer targets gross parsing faults such as hangs caused by malformed frames (not yet demonstrated), and logged seeds make campaigns reproducible. The setup follows none of the practices recommended in [19] (repetitions, statistics, baselines), and the scoring scheme still needs validation (Section III-G). In a secure-development workflow the platform is meant as a cheap pre-deployment check of input-handling code, not as a replacement for coverage-guided fuzzing, static analysis, or review.

VIII. Limitations and Future Work

A. Current Limitations

- Only UART is implemented. SPI and I2C are stubs, and CAN needs an external transceiver; the CAN RX pin (GPIO25) conflicts with the heartbeat input in the repository wiring table.
- The DUT must implement the reference frame protocol and drive the heartbeat pin, and the oracle detects only liveness loss and response errors.
- Seeded-fault sensitivity was not evaluated, and no baselines or repeated trials exist. The OLED entry for the Custom profile duration is listed as future work in the repository.

B. Future Work

The most valuable next step is seeded-fault validation, which is planned but not yet done. Reference DUT variants would each contain one fault: (i) a missing payload-length check before copying into a fixed-size buffer, (ii) incorrect handling of checksum errors, (iii) a parser that fails on shortened frames, and (iv) an input-dependent fault. Against a random-input baseline, the study would measure detection rate, time to detection, replay rate, minimization rate, false-positive rate (FAIL on a fault-free DUT), and false-negative rate (no detection despite a fault). The repository roadmap also lists SPI/I2C fuzzing, adaptive and state-aware mutation, logic-analyzer correlation (module built, results not analyzed or integrated), and more target microcontrollers.

IX. Conclusion

This paper presented AutoFuzzer, a standalone ESP32-based platform for UART robustness testing of embedded firmware, motivated by the security risk of malformed-input handling. It combines seven primary mutation classes, phase-based scheduling, deterministic replay, heartbeat- and response-based failure detection, and an on-device interface. The evaluation comprises 45 passing host-side tests and one recorded run (258 packets, 14 NACKs, no failures, 75/100) that matches the firmware scoring rules; the target and raw log of that run are unavailable, and the host tests exercise a Python reference implementation rather than the compiled ESP32 firmware. The two README-reported campaigns and the 41 archived reports cannot be independently verified. The current evaluation therefore supports the implementation and reporting pipeline only, not the platform's fault-detection effectiveness, and seeded-fault experiments are the planned next step.

ACKNOWLEDGMENT

The authors would like to thank their faculty adviser, Dr. Rajesh Kannan, for his suggestions and guidance in this project, as well as the Department of Electronics and Communication Engineering, Amrita Vishwa Vidyapeetham, Amritapuri campus, for providing laboratory facilities in the subject of Embedded Systems (23ECE313).

REFERENCES

[1] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of UNIX utilities," *Commun. ACM*, vol. 33, no. 12, pp. 32–44, Dec. 1990, doi: 10.1145/96267.96279.

[2] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2312–2331, Nov. 2021, doi: 10.1109/TSE.2019.2946563.

[3] X. Zhang, C. Zhang, X. Li, Z. Du, B. Mao, Y. Li, Y. Zheng, Y. Li, L. Pan, Y. Liu, and R. H. Deng, "A survey of protocol fuzzing," *ACM Comput. Surv.*, vol. 57, no. 2, Art. no. 35, pp. 1–36, 2025, doi: 10.1145/3696788.

[4] M. Muench, J. Stijohann, F. Kargl, A. Francillon, and D. Balzarotti, "What you corrupt is not what you crash: Challenges in fuzzing embedded devices," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2018, doi: 10.14722/ndss.2018.23166.

[5] Asmita, R. Tsang, S. Ghimire, S. Salehi, and H. Homayoun, "Bare-metal firmware fuzzing: A survey of techniques and approaches," *IEEE Access*, vol. 13, pp. 98253–98277, 2025.

[6] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNet: A grey-box fuzzer for network protocols," in *Proc. IEEE Int. Conf. Softw. Testing, Verification and Validation (ICST)*, 2020, pp. 460–465, doi: 10.1109/ICST46399.2020.00062.

[7] A. A. Clements, E. Gustafson, T. Scharnowski, P. Grosen, D. Fritz, C. Kruegel, G. Vigna, S. Bagchi, and M. Payer, "HALucinator: Firmware re-hosting through abstraction layer emulation," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1201–1218.

[8] B. Feng, A. Mera, and L. Lu, "P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1237–1254.

[9] M. Bley, T. Scharnowski, S. Wörner, M. Schloegel, and T. Holz, "Protocol-aware firmware rehosting for effective fuzzing of embedded network stacks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Taipei, Taiwan, Oct. 2025, doi: 10.1145/3719027.3765125.

[10] A. Mera, C. Liu, R. Sun, E. Kirda, and L. Lu, "SHiFT: Semi-hosted fuzz testing for embedded applications," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, 2024, pp. 5323–5340.

[11] F. Hofhammer, Q. Wang, A. Bhattacharyya, M. Salehi, B. Crispo, M. Egele, M. Payer, and M. Busch, "EmbedFuzz: High speed fuzzing through transplantation," arXiv:2412.12746, Dec. 2024.

[12] W. Li, J. Shi, F. Li, J. Lin, W. Wang, and L. Guan, "μAFL: Non-intrusive feedback-driven fuzzing for microcontroller firmware," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, doi: 10.1145/3510003.3510208.

[13] M. Eisele, D. Ebert, C. Huth, and A. Zeller, "Fuzzing embedded systems using debug interfaces," in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2023, pp. 1031–1042.

[14] M. Borsig, S. Nitzsche, M. Eisele, R. Gröll, J. Becker, and I. Baumann, "Fuzzing framework for ESP32 microcontrollers," in *Proc. IEEE Int. Workshop Inf. Forensics Secur. (WIFS)*, 2020, pp. 1–6.

[15] F. Meisel, C. Spang, D. Volz, and A. Koch, "TaPaFuzz: Hardware-accelerated RISC-V bare-metal firmware fuzzing using rapid job launches," *J. Syst. Archit.*, vol. 156, Art. no. 103288, Nov. 2024, doi: 10.1016/j.sysarc.2024.103288.

[16] M. Salehi, D. Hughes, and B. Crispo, "μSBS: Static binary sanitization of bare-metal embedded devices for fault observability," in *Proc. Int. Symp. Research in Attacks, Intrusions and Defenses (RAID)*, 2020.

[17] T. Werquin, R. Hubrechtsen, A. Thangarajan, F. Piessens, and J. T. Mühlberg, "Automated fuzzing of automotive control units," arXiv:2102.12345, 2021.

[18] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, Feb. 2002, doi: 10.1109/32.988498.

[19] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018, pp. 2123–2138, doi: 10.1145/3243734.3243804.

[20] G. Marsaglia, "Xorshift RNGs," *J. Stat. Softw.*, vol. 8, no. 14, 2003, doi: 10.18637/jss.v008.i14.